FIREBRAND
LYCEUM EDUCATION GROUP

Module 4: Software Design
Session 3 stretch: where these techniques come from

Software Developer Level 4



Use case goal levels (Cockburn)

The idea

Alistair Cockburn sorts use cases by the level of the goal.

Summary: a long-running goal, such as getting a home repaired.

User goal: what one person sets out to do, with a result they care about, such as reporting a repair online.

Subfunction: a step inside a user goal, such as uploading a photo.

Why it matters

Use cases at the wrong level make a set unreadable: dozens of subfunctions and no clear goals.

Try this: is 'Report a repair online' a user goal? Name one summary goal above it and one subfunction below it.

FIREBRAND
LYCEUM EDUCATION GROUP

INVEST (Bill Wake, 2003)

USER STORIES

> **A good story is Independent, Negotiable, Valuable, Estimable, Small and Testable.**

Why it matters: each letter names a way a story fails in planning, building or acceptance, so the weakest letter tells you what to fix first.

Try this: take one story you wrote in Session 3. Which letter is weakest, and what single change would fix it?

FIREBRAND
LYCEUM EDUCATION GROUP

Behaviour-driven development

Dan North described behaviour-driven development (BDD) in 2006: talk about the behaviour a system should have, not about tests. Given/When/Then grew out of that work as a way to write acceptance criteria in business language that can also run as automated tests.

Why it matters: when the criterion and the automated test are the same sentence, they cannot drift apart as the code changes.

Try this: rewrite one of your Session 3 criteria so that a tool could run it. Name the exact data in the Given and the exact observable result in the Then.

FIREBRAND
LYCEUM EDUCATION GROUP

Boundary values and partitions

Boundary value analysis and equivalence partitioning, from the ISTQB Foundation syllabus.

1. **Partition the inputs**
   Group values the system should treat the same, such as 1 May to 30 September and 1 October to 30 April for no heating at all.

2. **Test the edges**
   Errors cluster where groups meet. Two-value analysis tests each boundary and its nearest neighbour across the line.

3. **Why it matters**
   One test per group plus the edges finds most off-by-one defects with few tests.

4. **Try this**
   The damp and mould investigation target is 336 hours in this module. List the groups and the values you would test.

The Three Amigos

Business, development and testing read a story together before it is built.

George Dinwiddie's name for this means at least three viewpoints, not exactly three people.

Why it matters: each view finds different gaps: what is needed, how it works, what could go wrong.

Try this: for your safety criterion, write the one question each amigo would ask.



FIREBRAND
LYCEUM EDUCATION GROUP

The cost of late defects (contested)

THE CLASSIC CLAIM

A defect fixed in production costs 100 times more than one fixed in requirements.

Traced to Barry Boehm's data (1976, 1981).

Often repeated without a source.

WHAT THE EVIDENCE SUPPORTS

Finding defects sooner is usually cheaper; the multiplier is disputed.

Small systems show a far flatter curve.

Try this: what made a late defect you have seen costly, the phase or the delay? Keep it general.

FIREBRAND
LYCEUM EDUCATION GROUP



Optional self-study for after Session 3. You do not need it for the Consolidate Quiz.

How to use it: one theory per slide, about 5 minutes each, so about 30 minutes in total. Each slide gives the idea in plain words, why it matters for a developer, and a Try this question that applies it to Kestrel Moor Homes or to your own work. The notes under each slide repeat the idea and give the full reference so you can read further.

The six theories: Cockburn's use case goal levels; INVEST for user stories; behaviour-driven development and Given/When/Then; boundary value analysis and equivalence partitioning; the Three Amigos conversation; and the rising cost of late defects, which is contested because the classic figures are disputed.

Bring your Try this answers to the start of Session 4 or to a coaching session if you want feedback.

Keep workplace examples general: no personal or customer data, credentials or confidential material, and no employer data in unapproved AI tools.

Use case goal levels (Cockburn)

The idea

Alistair Cockburn sorts use cases by the level of the goal.

Summary: a long-running goal, such as getting a home repaired.

User goal: what one person sets out to do, with a result they care about, such as reporting a repair online.

Subfunction: a step inside a user goal, such as uploading a photo.

Why it matters

Use cases at the wrong level make a set unreadable: dozens of subfunctions and no clear goals.

Try this: is 'Report a repair online' a user goal? Name one summary goal above it and one subfunction below it.

FIREBRAND LYCEUM EDUCATION GROUP

2

About 5 minutes of self-study.

The idea: Alistair Cockburn sorts use cases by the level of the goal. A summary goal is long-running, such as getting a home repaired. A user goal is what one person sets out to do and comes away with a result, such as reporting a repair online. A subfunction is a step inside a user goal, such as uploading a photo.

Why it matters: a set of use cases at the wrong level is hard to read. Dozens of subfunctions hide the goals people actually have, and one huge summary use case hides the decisions. Asking "why does the actor do this?" moves you up a level; asking "how?" moves you down.

Try this: is "Report a repair online" a user goal? Name one summary goal above it and one subfunction below it. A useful test for a subfunction: would anyone feel they had achieved something after doing only that?

Reference: Alistair Cockburn, Writing Effective Use Cases (Addison-Wesley, 2001). The book describes the summary, user-goal and subfunction levels, and suggests writing a main success scenario of 3 to 9 steps.

INVEST (Bill Wake, 2003)

USER STORIES

> **A good story is Independent, Negotiable, Valuable, Estimable, Small and Testable.**

Why it matters: each letter names a way a story fails in planning, building or acceptance, so the weakest letter tells you what to fix first.

Try this: take one story you wrote in Session 3. Which letter is weakest, and what single change would fix it?

FIREBRAND LYCEUM EDUCATION GROUP

3

About 5 minutes of self-study.

The idea: INVEST is a checklist for a good user story: Independent, Negotiable, Valuable, Estimable, Small and Testable. It is not a score; it helps you find the letter a story is weakest on.

Why it matters: each letter names a way a story fails. A story that is not Small will not finish in a sprint; one that is not Testable cannot be accepted; one that is not Independent waits on other work. Fixing the weakest letter first is usually the quickest improvement.

Try this: take one story you wrote in Session 3. Which letter is weakest, and what single change would fix it? A good answer names the letter, quotes the weak part of the story and gives the rewritten line.

Reference: Bill Wake, "INVEST in Good Stories, and SMART Tasks", xp123.com, August 2003. Wake also mentions Ron Jeffries' three Cs of a story: card, conversation and confirmation.

Behaviour-driven development

Dan North described behaviour-driven development (BDD) in 2006: talk about the behaviour a system should have, not about tests. Given/When/Then grew out of that work as a way to write acceptance criteria in business language that can also run as automated tests.

Why it matters: when the criterion and the automated test are the same sentence, they cannot drift apart as the code changes.

Try this: rewrite one of your Session 3 criteria so that a tool could run it. Name the exact data in the Given and the exact observable result in the Then.

FIREBRAND LYCEUM EDUCATION GROUP

4

About 5 minutes of self-study.

The idea: Dan North described behaviour-driven development (BDD) as talking about the behaviour a system should have rather than about tests. The Given/When/Then template grew out of that work: it states a story's acceptance criteria in business language that a tool can also run as an automated test.

Why it matters: when the criterion and the automated test are the same sentence, they cannot drift apart as the code changes, and a failing test points straight at the behaviour that broke.

Try this: rewrite one of your Session 3 criteria so that a tool could run it. Put exact data in the Given (a date, a household), one action in the When, and something a tester can observe in the Then. Tools such as Cucumber match each line to a step definition in code, but you do not need a tool for this exercise.

References: Dan North, "Introducing BDD", first published in Better Software magazine in March 2006 and republished on dannorth.net. The Cucumber project's "History of BDD" page describes how the Given/When/Then template was developed and credits the As a / I want / So that story template to Rachel Davies at Connextra.

Boundary values and partitions

Boundary value analysis and equivalence partitioning, from the ISTQB Foundation syllabus.

1. **Partition the inputs**
   Group values the system should treat the same, such as 1 May to 30 September and 1 October to 30 April for no heating at all.
2. **Test the edges**
   Errors cluster where groups meet. Two-value analysis tests each boundary and its nearest neighbour across the line.
3. **Why it matters**
   One test per group plus the edges finds most off-by-one defects with few tests.
4. **Try this**
   The damp and mould investigation target is 336 hours in this module. List the groups and the values you would test.

FIREBRAND LYCEUM EDUCATION GROUP

5

About 5 minutes of self-study.

The idea: equivalence partitioning groups input values that the system should treat the same way, so one test can stand for each group. Boundary value analysis then tests where the groups meet, because that is where off-by-one mistakes cluster. Two-value boundary analysis tests each boundary value and its nearest neighbour across the line; three-value analysis adds the neighbour on the other side too.

Why it matters: one test per group plus the edges finds most off-by-one defects with very few tests.

Try this: in this module the damp and mould investigation target is 336 hours from the report. List the groups, including one invalid group, and the values you would test. Say what precision the data records times in, because that decides the nearest neighbour.

Reference: ISTQB Certified Tester Foundation Level syllabus, version 4.0.1 (2024), section 4.2, black-box test techniques, which covers equivalence partitioning and two-value and three-value boundary value analysis. The syllabus is free on istqb.org.

The Three Amigos

Business, development and testing read a story together before it is built.

George Dinwiddie's name for this means at least three viewpoints, not exactly three people.

Why it matters: each view finds different gaps: what is needed, how it works, what could go wrong.

Try this: for your safety criterion, write the one question each amigo would ask.



FIREBRAND LYCEUM EDUCATION GROUP

6

About 5 minutes of self-study.

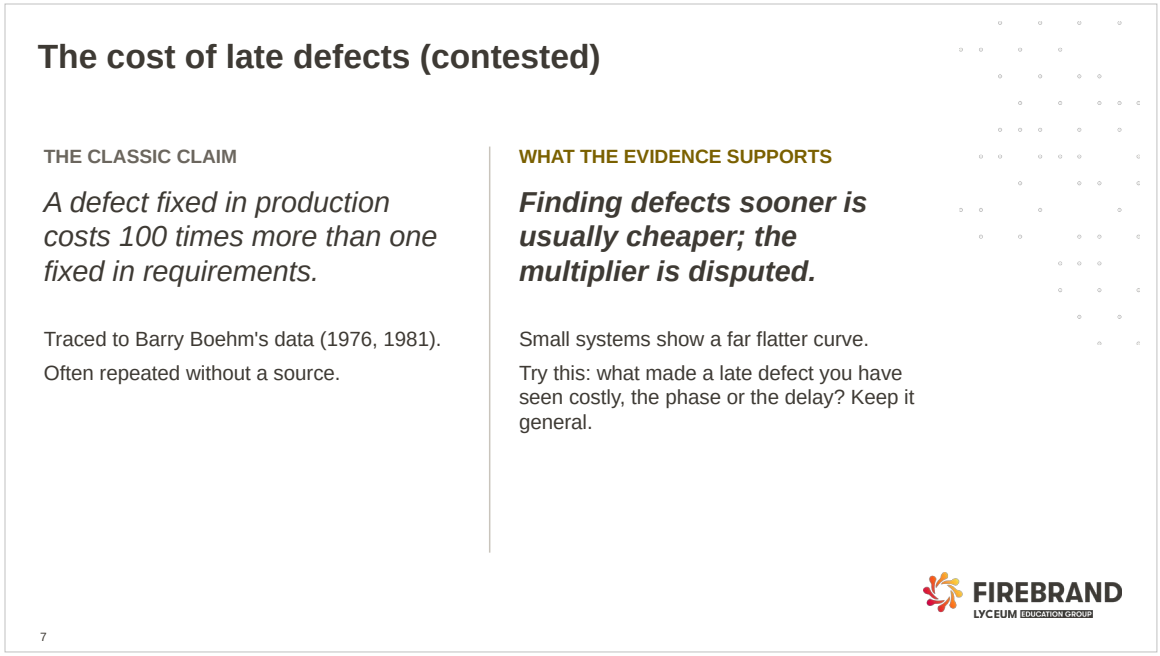
The idea: before a story is built, someone from the business, a developer and a tester read it together. Each view finds different gaps: what is needed, how it will work and what could go wrong. George Dinwiddie, who writes about the practice, says the name means at least three viewpoints, not exactly three people; user experience or security specialists sometimes join.

Why it matters: most missing requirements are found by asking the question nobody in the room thought to ask. Three viewpoints make that much more likely before any code is written.

Try this: for your safety criterion from Session 3, write the one question each amigo would ask. Then add a fourth viewpoint, such as the contact centre, and say what it would ask.

Reference: George Dinwiddie, "The Three Amigos Strategy of Developing User Stories", StickyMinds, April 2014.

The photo is an illustrative generated image of a fictional team.



About 5 minutes of self-study. This theory is contested: take the direction from it, not the numbers.

The idea: the classic claim is that a defect fixed after release costs about 100 times more than one fixed during requirements. The figure goes back to Barry Boehm's data from large projects in the 1970s and 1980s and has been repeated many times, often without a source. Later work found the multiplier depends heavily on the kind of system.

Why it matters: finding a defect sooner is usually cheaper, which is why teams agree testable criteria before the build. But the size of the saving varies, so be wary of anyone quoting one precise multiplier.

Try this: think of a late defect you have seen. What made it costly: the phase it was found in, or the delay between making the mistake and finding it? Keep it general.

References: Barry Boehm, Software Engineering Economics (Prentice-Hall, 1981). Barry Boehm and Victor Basili, "Software Defect Reduction Top 10 List", IEEE Computer, January 2001, which says finding and fixing a problem after delivery is often 100 times more expensive, but that for small, non-critical systems the factor is more like 5 to 1. Laurent Bossavit, The Leprechauns of Software Engineering (2015), traces how the cost-of-change figures spread. The Register (July 2021) reported that the widely copied chart credited to an "IBM Systems Sciences Institute" could not be traced to an original study.

Keep workplace examples general: no personal or customer data, credentials or confidential material, and no employer data in unapproved AI tools.